

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的动态防
控技术研究

学 号： M202310652

研 究 生： 韩晓阳

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的动态防控技术研究

**Research on Dynamic Prevention and Control
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：韩晓阳

指导教师：孙昌爱

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate: Xiaoyang Han

Supervisor: Chang'ai Sun

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的动态防控技术研究

研究生: 韩晓阳

指 导 教 师: 孙昌爱 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: 单位: 职称:

 单位: 职称:

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

论文中文摘要字数约为 300~600 字，如遇特殊需要字数可以略多，限一页。

论文摘要是论文内容不加注释和评论的简短陈述，一般以第三人称语气写成。摘要的编写应遵循下列原则：

1) 摘要应具有独立性和自含性，即不阅读论文的全文，就能获得必要的信息。摘要是学位论文的缩影，是学位论文的主要内容、见解、结论简短明了的缩写。

2) 摘要应是一篇完整的短文，可以独立使用，可以引用。

3) 摘要的内容应包含与论文等量的主要信息，供读者确定有无必要阅读全文，也可供文摘汇编等二次文献采用。

4) 摘要一般应说明研究工作的目的意义、研究方法、研究结果、主要结论及意义、创造性成果和新见解，而重点是结论和创新点。

5) 要用文字表达，不要附图和照片，除了实在无变通办法可用以外，摘要中不用图、表、化学结构式、非公知公用的符号和术语，不要使用表格、公式、上下标以及其他特殊符号，要突出重点，阐述清楚，少用数据表。论文摘要用语力求简洁、准确。原则上 300~600 字。

关键词：摘要；论文；要求；字数；格式（关键词个数为 3~5 个，正式写作请删除此括号）

Research on Dynamic Prevention and Control Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

In environmental economics, environmental resources including environmental quality are categorized as amenity resources. Due to its importance to human welfare, the amenity resources theoretical study and valuation is an ongoing issue at the academic frontier in the environmental economics area.

注：论文的英文摘要应有英文题目和关键词，内容与中文摘要相同，用另页置于中文摘要之后；外文摘要实词在 300 个左右。

Key Words: Key word 1; Key word 2; Key word 3; ...

1 引言

1.1 背景与意义

Kubernetes 已成为云原生基础设施的事实标准，并被广泛用于承载容器化应用的部署、调度与弹性伸缩等关键能力 [1–3]。随着云原生生态不断扩展，大量第三方组件（监控、服务网格、CI/CD 等）以 Sidecar、Controller 或 Operator 等形式接入集群，其运行往往依赖 ServiceAccount 与 RBAC 权限配置来访问或操作集群资源。由于权限边界复杂、配置高度异构且随业务迭代频繁变更，权限配置不当容易引入过度授权与越权访问风险，进一步触发敏感资源泄露或未授权操作等安全事件。

从权限视角看，Kubernetes 基于扩展 RBAC 机制对资源访问进行统一管理 [1, 4]，而最小权限原则则为权限配置治理提供了通用的安全基线 [5]。然而，在实际工程中，第三方应用的权限需求往往难以被精确刻画，导致“为保证可用而放宽权限”的做法普遍存在，从而产生冗余权限并扩大攻击面 [6, 7]。与此同时，权限提升攻击还可能借助配置缺陷、DevOps 流水线以及运行时行为实现权限扩张，使得仅依赖静态规则的检测方法在动态环境下容易出现适应性不足与精度受限等问题 [8, 9]。

在漏洞响应层面，高危漏洞的补丁发布、版本验证与生产升级通常存在时间差；当补丁不可用或难以及时部署时，生产环境需要先采用可快速下发、可验证且可回滚的临时防控措施，以尽可能降低风险暴露面并抑制攻击链扩散。基于上述现实需求，本文面向云原生应用权限提升漏洞，聚焦在补丁窗口期内的动态防控：以证据可追溯的上下文为输入，自动生成并筛选可交付的临时防控策略，从而提升集群的抗攻击能力与运维可控性。

1.2 内容与成果

为应对云原生权限提升漏洞的临时防控需求，本文主要工作与贡献如下：

- 提出以 layer/method/actions 为核心的结构化策略表示与 JSON schema 约束，结合证据来源标注，确保策略可检查、可复核。
- 构建云原生开源应用知识库与检索增强上下文，支持可追溯的证据构建与漏洞关联。

- 设计“RAG+CoT+Feedback”分阶段策略生成流程，将漏洞分析、控制手段选型与评审优化显式化，降低噪声与误报。
- 建立门槛评分与三维质量向量（有效性、无干扰性、可部署性）的评价体系，对候选策略进行筛选与排名，并通过实验验证各模块的贡献。

1.3 论文组织结构

全文结构如下：第2章介绍相关概念、技术与国内外研究现状；第3章给出面向云原生权限提升漏洞的智能防控技术体系；第4章描述漏洞防控系统设计实现；第5章呈现实验设置与结果；第6章给出案例分析；第7章总结与展望。

2 背景介绍

2.1 相关概念与技术

2.1.1 容器

容器（Container）是一种以**镜像（Image）**为交付单元、以操作系统内核能力为隔离基础的轻量级运行环境。与传统虚拟机通过虚拟化硬件来隔离不同，容器更强调**进程级隔离与环境一致性**：应用及其依赖以镜像形式打包交付，运行时由容器引擎创建隔离的进程视图与资源配额，再启动容器内的目标进程。

从实现机制看，容器的关键能力主要来自三类内核与运行时支撑：**（1）命名空间（Namespace）隔离**，将进程、网络、挂载点、IPC、用户等资源视图隔离开来，使容器内进程“看见”的系统环境与宿主机相互独立；**（2）控制组（cgroup）资源管控**，对 CPU、内存、I/O 等资源进行限制与统计，防止单个工作负载挤占宿主机资源；**（3）镜像与分层文件系统**，将应用与依赖固化为可复用的只读层，并基于 OverlayFS 技术叠加读写层形成运行时文件系统，从而同时实现复用、缓存与快速分发。

在工程实践中，容器镜像的可复用性与快速分发构成云原生交付的关键能力，但亦伴随若干显著的安全隐患。首先，镜像可能包含过期依赖或不当默认配置。长期未更新的基础镜像会导致已知漏洞残留；容器若以 `root` 身份运行，或在镜像中嵌入调试工具与敏感凭据，则会扩大被利用的风险。其次，容器的隔离能力取决于宿主机内核与运行时配置。启用特权模式、挂载宿主机路径、放宽 Linux capabilities，或禁用 `seccomp` 与 `AppArmor` 等安全机制，均会削弱容器与宿主机之间的隔离，从而使攻击面从容器内部向宿主机乃至整个集群扩展。鉴于上述风险，容器安全治理应覆盖镜像构建阶段的依赖管理与基线加固、制品交付环节的制品管理与签名，以及运行时的权限最小化与最小暴露等生命周期环节，并在各阶段引入自动化检测与策略约束以降低总体风险。

2.1.2 Kubernetes：架构与工作原理

Kubernetes 是面向容器工作负载的编排系统，其核心思想是以**声明式 API** 描述期望状态（desired state），再由控制平面持续驱动实际状态（actual state）向期望状态收敛 [1]。这一“**控制回路（reconcile loop）**”使得部署、扩缩容、故障自愈与滚动升级等运维动作可以自动化、可重复且具备可观测性。

从整体架构看，Kubernetes 由**控制平面（Control Plane）**与**工作节点（Worker Node）**两部分构成。控制平面负责提供 API、调度与集群控制逻辑，其中：**API Server** 是所有资源访问与状态变更的统一入口，承担认证、授权、准入与资源对象持久化等关键职责；**调度器（Scheduler）** 根据资源请求、亲和/反亲和、污点与容忍等约束为待调度 Pod 选择节点；各类**控制器（Controller）**（如 Deployment/ReplicaSet、Job、Node、Endpoint 等控制器）基于资源对象的当前状态触发协调动作，形成“观察—决策—执行”的闭环。工作节点侧，**kubelet** 负责与 API Server 通信并将 Pod 规格落实到本机运行时，**容器运行时（Container Runtime）** 负责拉取镜像、创建容器并管理生命周期，**kube-proxy**（或由 CNI/eBPF 方案替代）负责服务转发与网络规则的实现。

围绕上述组件，Kubernetes 进一步抽象出一组核心对象以承载不同层面的运维语义：**Pod** 是最小调度单元，包含一个或多个共享网络与存储命名空间的容器；**Deployment/StatefulSet/DaemonSet** 分别面向无状态、有状态与节点守护型工作负载提供期望副本与更新策略；**Service** 与 **Endpoint/EndpointSlice** 提供稳定的服务发现与负载均衡抽象；**Ingress**（或 Gateway API）面向南北向流量提供入口控制；**ConfigMap/Secret** 用于配置与敏感信息的声明式注入；**Volume/PV/PVC** 则提供存储生命周期与绑定关系的抽象。

Kubernetes 的工作原理可以概括为“**资源对象驱动的状态机**”。当用户提交 YAML 清单或通过控制器创建资源对象时，请求首先进入 API Server：经过认证、授权与准入控制后，资源对象被持久化并成为集群事实状态。随后，控制器与调度器通过监听资源变化（watch）获取事件：调度器为 Pending 的 Pod 选择节点并写回绑定结果；对应节点上的 kubelet 读取 PodSpec，调用容器运行时拉取镜像、创建容器，配置网络与挂载卷，并持续上报状态与探针结果。一旦发生节点故障、容器退出或副本数变化，控制器会再次触发协调动作，保证系统最终收敛到期望状态。

为便于理解控制平面与工作节点之间的职责划分与交互关系，如图2-1所

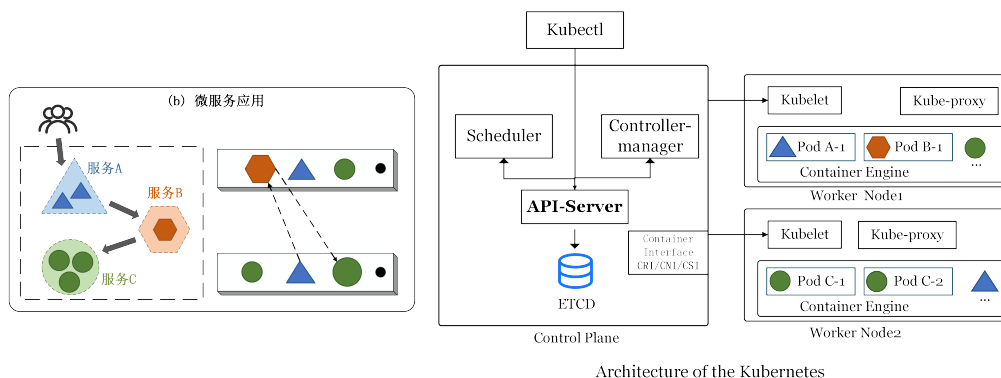


图 2-1: Kubernetes 体系结构示意图

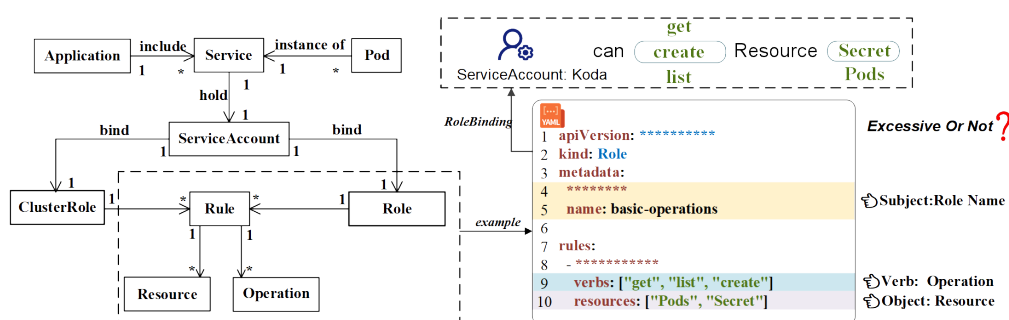


图 2-2: Kubernetes RBAC 权限配置 (YAML) 示意 [10]

示给出了 Kubernetes 的体系结构示意图。

2.1.3 Kubernetes 权限模型

Kubernetes 将“资源访问”统一收敛到 API Server，使得安全治理的关键落在身份（Authentication）与授权（Authorization）链路之上。其中，RBAC（Role-Based Access Control）是最核心的授权模型之一 [1, 4]。在典型流程中，工作负载（Pod）以 ServiceAccount 作为身份主体；权限以 Role/ClusterRole 的形式被抽象为若干规则（rules），规则通常以“API 组与资源类型（api-Groups/resources）—操作动词（verbs）—作用域（namespace/cluster）”等维度描述允许的操作集合；随后通过 RoleBinding/ClusterRoleBinding 将主体与角色绑定，使得主体获得对特定资源的访问能力。与之配合，Namespace 用于隔离租户与资源域，NetworkPolicy 等机制用于收敛网络可达性并减少横向移动空间。

在落地层面，RBAC 通常以 YAML 清单进行声明式配置：通过编写 Role/ClusterRole 的规则集合，再以（Cluster）RoleBinding 将其授予 ServiceAccount 等主体。图2-2给出了权限配置（YAML）的示意，用于直观展示“规则—绑定—主体”的授权链路。

在工程实践中，第三方组件（如 Prometheus、Jenkins、Istio 等）接入往往需要访问多类资源与 API，对权限配置的依赖更强；一旦存在**过宽的 RBAC 绑定或错误的身份边界**（例如将集群级角色绑定到不必要的 ServiceAccount），便可能形成权限滥用与权限提升的入口。因此，理解从“主体—绑定—角色规则—资源对象”的授权链路，以及其与命名空间隔离、网络可达性的耦合关系，是后续分析权限提升漏洞成因与制定可落地防控措施的基础。

2.1.4 Kubernetes 权限提升漏洞

权限提升（Privilege Escalation）指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置，将自身权限提升到更高等级的过程。在 Kubernetes 场景下，权限提升既可能表现为授权范围扩大（例如能够访问更多 Kubernetes API，这通常由 RBAC 规则决定），也可能表现为节点侧系统权限提升（例如获得节点上的 root 权限），从而对更大范围的资源与工作负载产生影响。

在云原生环境中，权限提升的危害往往具有更强的放大效应。一方面，单个节点通常承载多个 Pod，节点侧提权成功后，攻击者可能对同节点上的多个工作负载实施窃取敏感信息、篡改运行环境或注入恶意组件等行为。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化；当攻击者获得更高权限后，可能进一步读取 Secret、创建高权限工作负载、修改网络策略或部署守护进程，进而将影响从单一应用扩展到集群层面的安全与稳定。因此，权限提升漏洞不仅是单点缺陷，还可能引发平台级安全事件与隔离失效。

为说明节点侧提权在云原生场景中的现实危害，图2-3给出了 CVE-2024-33522 的示例。该漏洞影响 Calico 的 CNI 安装程序：在部分易受影响版本中，安装二进制的 SUID（Set User ID）位配置不当，同时允许攻击者控制输入二进制。在攻击者已具备 Kubernetes 节点本地访问的前提下，攻击者可诱导安装过程以更高权限执行任意二进制，从而实现节点侧权限提升。虽然该漏洞的利用前提包含节点本地访问，但在真实环境中，这一前提可能由其他入侵路径满足（例如节点服务暴露、弱口令、供应链投毒，或通过其他漏洞获得节点执行能力）。一旦节点侧权限提升成功，攻击者可能读取或篡改节点关键数据（如容器运行时目录、配置与日志），窃取工作负载凭据与令牌，干扰网络组件行为，并进一步扩大影响范围，从而显著加剧集群整体风险。

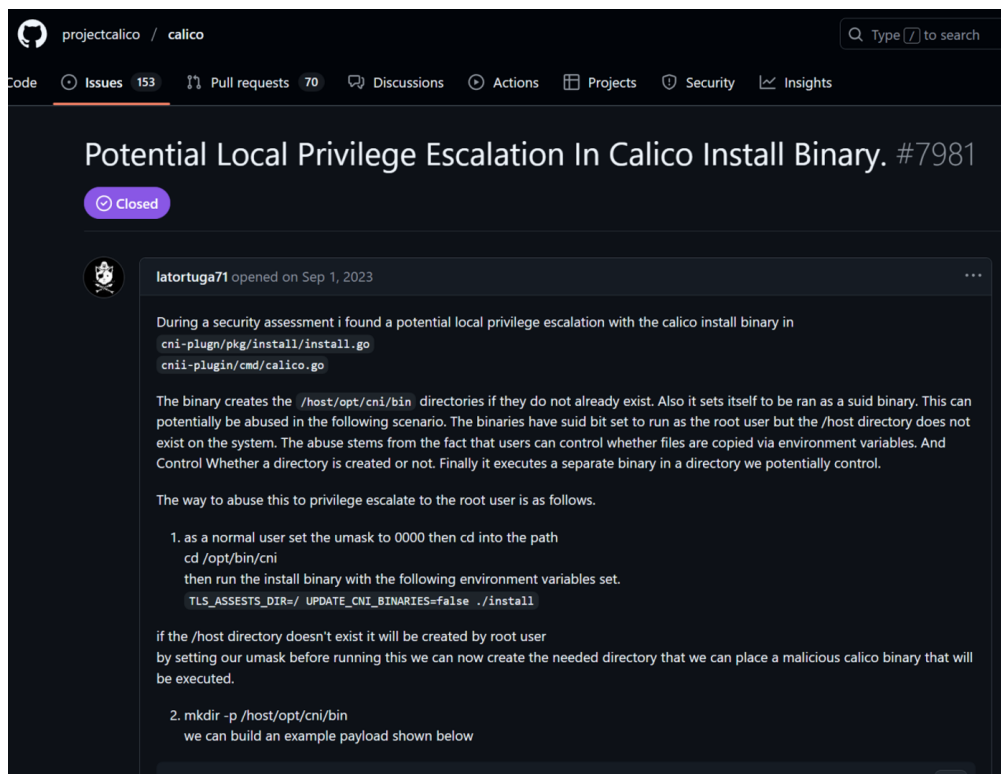


图 2-3: CVE-2024-33522 (Calico) 节点侧权限提升漏洞报告 ISSUE

2.2 云原生安全工具

在补丁未就绪或难以及时升级的阶段，生产环境往往需要依赖可快速下发、可验证且可回滚的控制手段来降低风险暴露面。从“上线前治理”到“运行时防护”的完整链路来看，云原生安全工具通常覆盖四个互补层面：（1）运行时检测与审计（在不中断业务的前提下对异常行为进行告警与取证）；（2）准入与策略约束（在资源进入集群前阻断高风险配置）；（3）网络隔离与最小连通（收敛工作负载间可达性以抑制横向移动）；（4）静态扫描与配置审计（在构建与交付阶段发现已知漏洞与弱配置）。下文分别介绍运行时检测工具 Falco、准入控制工具 OPA/Gatekeeper 与 Kyverno、网络隔离机制 NetworkPolicy，以及静态扫描与配置审计类工具。

2.2.1 容器安全技术：Falco

在容器技术与云原生基础设施广泛部署的背景下，传统基于主机或网络的入侵监测系统（IDS）逐渐暴露出适应性不足、粒度较粗以及缺乏云原生上下文语义等问题。为满足容器化环境对运行时安全的实时性与上下文感知需求，开源项目 Falco 应运而生。Falco 最初由 Sysdig 公司开发，并于 2018 年捐赠给云原生计算基金会（CNCF），成为较早面向容器运行时的行为监测系

统之一。其核心思想是在**系统调用层**进行透明监控，并通过规则语言定义安全策略，从而在不中断业务的前提下实现精细化威胁监测与审计。

Falco 的监测机制建立在对 Linux 系统调用的动态捕获之上，其中 **eBPF** 是其能力落地的关键支撑之一。**eBPF** (extended Berkeley Packet Filter) 允许在**内核态**运行经过验证的字节码程序，并以受控方式访问内核提供的上下文与辅助函数；**eBPF** 程序可动态挂载到系统调用入口/退出点、内核跟踪点 (tracepoint)、kprobe 以及网络相关钩子等位置，从而以较低侵入性捕获安全相关事件。相较于传统内核模块方案，**eBPF** 通过验证器与隔离执行机制降低了对内核稳定性的影响，并在性能与可观测性之间提供更可控的权衡，适合用于容器运行时的持续监测。

在事件捕获之后，Falco 在用户空间对 **eBPF** 输出的事件进行解析，并结合容器运行时与编排元信息，补齐云原生上下文（如容器 ID、Pod/Namespace、进程命令、文件路径等），最终形成高语义的行为事件流并触发告警/审计输出。该设计使 Falco 能够从“系统调用级信号”上升到“云原生对象级语义”，更贴近安全运营对定位与处置的需求。

规则系统是 Falco 的另一核心组成部分。不同于依赖模型训练或特征匹配的黑盒监测方法，Falco 采用面向安全运维的领域特定语言 (DSL) 来定义规则：每条规则通常由触发条件、约束表达式与输出字段组成。例如，可定义规则监控容器中是否出现交互式 Shell、是否访问关键系统文件、是否执行未经授权的二进制程序等。规则驱动方式便于安全团队结合业务场景进行定制，且可解释性强，便于合规审计与取证分析。得益于与 Kubernetes 元信息的集成，Falco 的告警也能够关联到 Pod、Namespace、ServiceAccount 等云原生语义实体，降低“只见系统调用、不见业务对象”的分析成本。

需要指出的是，Falco 仍存在一定局限性：其规则库的维护较依赖经验，难以覆盖未知攻击或高度隐蔽的行为模式；同时其主要面向系统调用层，对应用层协议滥用或内存空间攻击等的可观测性有限。因此，在工程实践中，Falco 往往与准入控制、网络隔离与最小权限治理等手段组合使用，以形成覆盖“配置—通信—运行时”的分层防线。

2.2.2 准入控制与策略约束：OPA/Gatekeeper 与 Kyverno

准入控制 (Admission Control) 属于**事前防护**机制，用于在资源对象写入 API Server 之前对其进行校验或变更，从源头阻断高风险配置进入集群。OPA

(Open Policy Agent) 是通用策略引擎, Gatekeeper 作为其在 Kubernetes 中的实现形态之一, 支持以约束 (Constraint) 与模板 (ConstraintTemplate) 的方式将策略下发到集群; 策略可覆盖特权容器、宿主机路径挂载、危险 Linux capabilities、镜像来源与签名、RBAC 绑定范围过宽等高风险项。Kyverno 则强调“以 YAML 写策略”的可用性, 除校验外也支持对资源进行变更 (mutate) 与生成 (generate), 适合将安全基线与平台治理要求固化为可复用规则。

在漏洞补丁尚不可用时, 准入策略尤其适用于**快速止血**: 通过强制最小权限、限制高危特性与对敏感对象的访问边界, 可显著压缩攻击者可利用的配置入口。同时, 准入策略的发布与回滚成本较低, 便于在不同业务环境中进行灰度验证与影响评估。

2.2.3 网络隔离与最小连通: NetworkPolicy

网络隔离用于降低横向移动与数据外传风险, 是云原生环境中与身份权限控制互补的关键手段。Kubernetes 的 NetworkPolicy 为声明式策略, 允许以 Pod/Namespaced 选择器与端口规则的形式, 定义**入站 (Ingress)**/**出站 (Egress)** 通信的允许集合, 从而将网络访问从“默认全通”收敛到“默认拒绝 + 按需放通”。在多租户或微服务体系中, 合理的 NetworkPolicy 可将控制域落到命名空间与工作负载级别, 减少单点被攻破后对整个集群的扩散影响。

需要注意的是, NetworkPolicy 的效果依赖底层网络插件 (CNI) 对策略的实现能力。常见 CNI (如 Calico、Cilium、Antrea 等) 均可提供对 NetworkPolicy 的支持; 其中部分实现还可在标准三/四层策略之外, 扩展更细粒度能力 (例如基于 eBPF 的高性能转发与可观测性、面向应用协议的七层策略等)。工程实践中通常结合业务依赖关系梳理通信图谱, 先对关键命名空间与高价值服务进行最小连通收敛, 再逐步扩展覆盖范围, 并配合观测与告警机制验证策略变更带来的连通性影响。

2.2.4 静态扫描与配置审计: 镜像、依赖与 IaC

与运行时检测相对, 静态扫描更偏向**事前发现与持续治理**: 在镜像构建、交付与部署前, 通过自动化检查提前识别已知漏洞、弱配置与合规风险, 避免高风险对象进入集群。

第一类是**镜像与依赖漏洞扫描**。该类工具通常解析镜像分层文件系统与软件包清单, 基于公开漏洞库对已知 CVE 进行匹配, 并输出风险等级、受影

响版本与修复建议。代表性工具包括 Trivy[11]、Grype/Anchore[12] 等；其中，Syft[13] 等工具可生成软件物料清单（SBOM），用于提升漏洞告警的可追溯性与供应链治理能力。在工程实践中，镜像扫描往往与 CI 流水线结合，在构建阶段阻断高危漏洞或不合规组件进入制品仓库。

第二类是 **Kubernetes 清单与基础设施即代码（IaC）扫描**。此类工具面向 YAML/Helm/Terraform 等配置，检查特权容器、宿主机路径挂载、过宽权限、明文密钥、缺失资源限制等常见风险，并可作为代码评审与准入策略的前置门禁。例如，kubelinter[14]、kubescape[15]、kube-score[16]、kubesecc[17]、Polaris[18] 等可对工作负载清单进行规则化检查；Checkov[19] 等可对 Terraform 等 IaC 模板进行安全基线与合规性验证。静态扫描的优势在于易于规模化推广与持续运行，但也可能因业务差异产生误报，因此通常需要结合组织基线与例外机制、以及准入与运行时告警进行闭环。

2.3 国内外研究现状

围绕 Kubernetes 第三方应用的安全风险，国内外研究大体可分为“配置与清单风险分析”“最小权限与过度授权治理”“权限提升与运行时攻击场景建模/检测”三个方向。

（1）**配置与清单风险分析**。Kubernetes 的工作负载与权限往往以 YAML 清单形式交付，配置的可复制性也使得错误配置在开源生态中具有传播性。已有实证研究对开源 Kubernetes manifests 中的安全误配置进行了系统统计与归因，揭示了过宽权限、特权容器、弱隔离等问题的普遍性 [8]。该方向的价值在于刻画“风险在现实生态中的分布”，但其分析通常基于静态规则或启发式检查，难以充分覆盖不同集群环境与应用运行时差异所带来的动态性。

（2）**最小权限与过度授权治理**。最小权限原则是权限配置治理的基础思想 [5]。在实际系统中，冗余权限会扩大攻击面并提高误用概率 [6]；在 Kubernetes 场景下，第三方应用的过度权限被证明可被利用以实现对接群关键资源的接管或扩大攻击影响 [7]。围绕“如何刻画应用的合理权限需求并识别过度授权”，已有工作从静态分析、配置推导与经验规则等角度展开探索（例如 EPSan[20]、KubeFence[21] 等），但在工程落地时仍面临权限需求随版本演进、环境差异导致的泛化困难，以及“可用性优先”带来的权限放宽等现实约束。

（3）**权限提升与运行时攻击场景建模/检测**。权限提升不仅来源于静态配

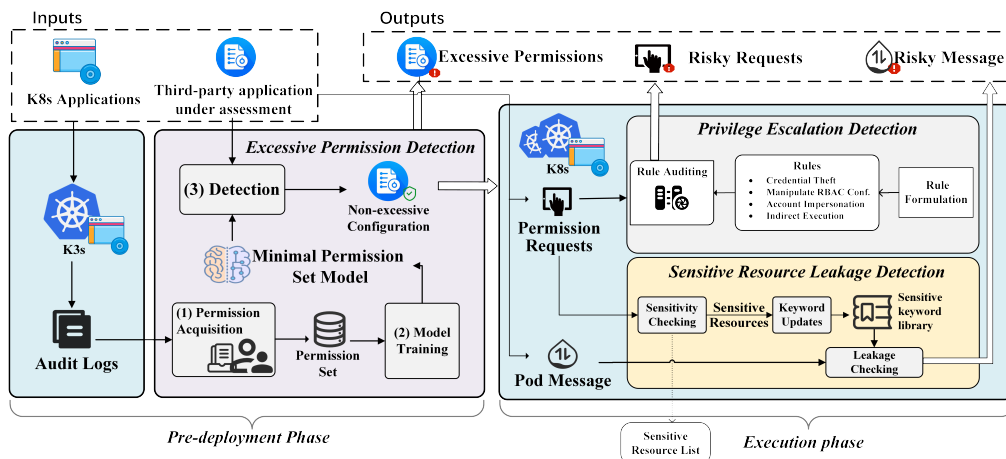


图 2-4: KubeGuard 框架示意 [10]

置本身，也与 DevOps 流水线、运行时 API 调用与权限链条有关。既有研究总结了 Kubernetes 权限提升的典型路径与攻击链条，并给出可复现的攻击场景与对抗视角 [7, 9]。该方向强调对“攻击如何发生”的机制理解，但将研究结论转化为补丁窗口期可直接采用的缓解策略，仍需要进一步解决“策略可部署/可回滚”“副作用可评估”“证据可追溯”等交付层问题。

在上述研究基础上，课题组工作 KubeGuard (ICWS 2025) 面向权限风险给出系统化检测框架，覆盖过度权限、权限提升与敏感信息泄露三类风险，并分别提出最小权限集构建、规则化审计与敏感词监测等机制 [10]。本文进一步聚焦于**权限提升类漏洞的补丁窗口期动态防控**：将多源证据组织为可追溯上下文，生成并筛选满足“可解释、可执行、可复核”的临时防控策略，以补足现有研究从“风险识别/机制分析”到“可交付缓解方案”的关键缺口。

2.4 动机与挑战

在云原生场景下，漏洞补丁的发布与升级往往存在时间差；同时，生产环境的组件组合与权限边界具有显著异构性。对于权限提升类漏洞而言，这意味着处置流程通常需要在“补丁尚不可用或难以及时部署”的阶段，先采取**临时防控措施**以降低风险暴露面。基于这一现实需求，本文将动机聚焦于可动态下发的安全控制域，通过准入控制、网络隔离与运行时检测等手段，对攻击链中的关键环节施加约束，实现可快速部署、可验证、可回滚的风险缓解。

为便于将上述动机具体化，本文将常见动态控制与工具能力归纳为如下可落地控制点：

- **准入与策略约束**：基于 OPA/Gatekeeper 或 Kyverno 等准入策略，对特

权容器、宿主机路径挂载、高风险 Linux capabilities、过宽 RBAC 绑定等高风险资源进行阻断或强制修正；

- **网络隔离与最小连通**：基于 NetworkPolicy 对命名空间与工作负载间通信进行白名单化，限制横向移动与数据外传路径；
- **运行时检测与响应**：基于 Falco 等运行时安全工具，监测异常系统调用、提权相关行为与逃逸迹象，并结合告警、隔离等处置流程降低影响扩散；
- **安全基线与最小权限**：结合 Pod 安全基线（如 Pod Security Standards）与账户/令牌治理，压缩默认配置下的权限暴露面。

面向权限提升漏洞的临时防控，自动化生成的方案需要同时满足三个要求：**可解释**（能够说明措施与风险点的对应关系）、**可执行**（给出可落地步骤，并明确验证与回滚边界）、**可复核**（关键结论可回溯到证据与环境假设）。然而，漏洞描述与运行环境信息往往存在噪声与缺失，不同控制手段对业务的副作用也存在差异，使得“生成—筛选—交付”难以依靠单一的启发式规则完成。为此，本文采用结构化产物、证据来源追踪与规则化质量评价相结合的方式，将策略生成过程约束到可检查的中间结果上，降低不确定性对交付质量的影响。

在上述约束下，引入大模型的核心动因在于提升对漏洞信息理解与策略生成的效率与覆盖度。面对公告、代码、讨论材料等多源非结构化证据，大模型可在检索增强上下文的支持下归纳漏洞成因，梳理权限边界与可控点，并生成多种候选的临时防控方案。与此同时，本文通过结构化字段设计、动作序列规范与门槛评价机制，将生成结果转化为可检查、可复核的交付产物，使其更契合 CVE 快速缓解与防控流程，并与必要的验证与审计环节保持一致。此外，本文在交付闭环中引入 human-in-the-loop（人工在环）机制：由安全工程师在门槛评价通过后对关键假设、证据引用、潜在业务副作用与回滚边界进行复核，并对安全策略部署决策作出最终确认。同时策略生成的每一步都可以人工检验中间结论与证据来源，并进行上下文操作和 prompt 优化等。

3 面向云原生权限提升漏洞的智能防控技术

本章给出本文方法体系：形式化定义、总体流程、证据构建、策略生成与优化以及质量评价。

3.1 形式化定义

3.1.1 基本对象

本节给出框架的形式化定义体系，用于精确刻画漏洞输入、策略生成与质量评价之间的关系。

定义 1 (漏洞实例): 令 $\mathcal{V}$ 表示漏洞实例集合，每个漏洞 $v \in \mathcal{V}$ 由三元组表示：

$$v = \langle \text{id}, d, m \rangle$$

其中 id 为 CVE 标识符， d 为 CVE 公告或其他文本描述， m 为元数据（如严重性、受影响组件等）。

定义 2 (安全上下文): 令 $\mathcal{C}$ 表示上下文集合，上下文 $c \in \mathcal{C}$ 是与漏洞 v 相关的证据性文本或结构化信息的集合。

在本框架中，上下文构建强调“证据来源可标注”，可将上下文表达为：

$$c = \{(t_i, \text{src}_i)\}_{i=1}^n$$

其中 t_i 为片段内容， src_i 为来源标识（例如文档路径、公告链接等）。

定义 3 (安全策略): 令 Π 表示安全策略空间，每个策略 $\pi \in \Pi$ 是一个结构化对象，包含三个核心字段：

$$\pi = \langle \text{layer}, \text{method}, \text{actions} \rangle$$

其中 layer 描述防控所在的技术层面（例如身份与权限、网络隔离、运行时检测等）， method 是防控方法的分类或摘要（例如网络隔离与认证强化）， actions 是具体可执行的防控操作序列，详见定义 3.1。

定义 3.1 (防控操作序列): 定义 actions 为一个有序的防控操作集合：

$$\text{actions} = [a_1, a_2, \dots, a_n]$$

其中每个操作 a_i 描述一个具体的防控步骤，可能涉及安全工具的使用或系统/集群配置的修改。为便于落地与复核，操作描述通常应包含：(1) 可执行的

步骤或命令；(2) 作用对象或组件（例如 NetworkPolicy、RBAC 配置等）；(3) 预期效果与验证方式；(4) 可能的副作用与回滚方案。

3.1.2 评价与选择

本节在统一符号与基本假设的前提下，形式化给出评分函数与排名算子的定义，以支撑后续跨方法、跨模型的可比性分析。

定义 4（单策略评分函数）：定义评分函数 $s : \Pi \rightarrow [0, 1]$ ，用于刻画策略是否满足最低某评价维度阈值。在临时防控语境下，评分函数的设计应关注可执行性与快速部署可行性而非永久修复，综合考虑策略的可部署性、有效性和副作用。

定义 5（三维度质量向量）：定义质量向量 $q(\pi)$ 为三维向量：

$$q(\pi) = (E(\pi), I(\pi), D(\pi))$$

其中 E 为有效性（Effectiveness：覆盖攻击面/阻断关键路径）， I 为无干扰性（Interference：对业务的副作用与误报风险越低越好）， D 为可部署性（Deployability：语法与步骤清晰、可操作）。

定义 6（多策略排名）：对候选集合 $\Pi_v \subseteq \Pi$ ，定义排名算子 R 输出一个排列：

$$R(\Pi_v, \text{dim}) \rightarrow (\pi_{(1)}, \pi_{(2)}, \dots, \pi_{(k)})$$

其中 $\text{dim} \in \{E, I, D\}$ 表示评价维度， $\pi_{(1)}$ 为该维度最优候选。“最佳策略”存在多目标权衡时排名比单一分数更适合呈现策略选择在不同维度下的权衡关系。

3.2 总体框架

本文方法的总体流程由三个阶段组成，依次完成证据构建、策略生成与质量筛选。整体输入为一个新披露的漏洞实例 v ；第一阶段将多源材料组织为可追溯的安全上下文 c ；第二阶段以 (v, c) 为输入生成并迭代优化候选策略集合 Π_v ；第三阶段对 Π_v 进行门槛检查与多维评价，筛选可交付策略并给出排序结果，为后续部署与处置决策提供依据。

1. **开源应用漏洞安全知识库构建：**收集目标开源应用的漏洞数据与相关材料（如代码仓库、官方文档、安全公告与社区讨论），检索并汇聚与漏洞相关的证据，形成带来源标注的安全上下文 c 。

2. **策略生成与优化**: 以 v 与 c 为输入, 在预定义的结构化约束下生成候选策略, 并结合评审反馈进行迭代优化, 得到候选集合 Π_v 。
3. **评价与质量控制**: 对 Π_v 中的候选策略执行门槛检查与多维质量评估, 基于评分与排名筛选可交付策略。

3.3 开源应用漏洞安全知识库构建

3.3.1 漏洞收集与分类框架

本节阐述漏洞样本的收集流程与两级成因分类框架的构建方法。该分类框架面向策略生成场景: 在输入侧提供相对稳定且可解释的风险语义线索, 用以约束模型的推理路径, 并辅助其对控制域与关键控制点进行选择, 从而提升输出策略的可部署性、可验证性与副作用可控性。

(1) 云原生应用清单与仓库映射。以 Kubernetes 及其生态开源应用为研究对象, 首先从 CNCF Landscape 获取云原生应用清单, 并对清单进行快照固化 (记录抓取日期与条目元数据) 以保证可复现性。随后将条目名称映射到其官方代码仓库 (如 GitHub URL), 形成后续漏洞关联与证据收集的目标集合。

(2) 漏洞收集与筛选。在数据源选择上, 综合使用 NIST NVD、MITRE CVE 官方库与安全公告平台 (如 GitHub Security Advisory、Google OSV 等), 检索 Kubernetes 及其生态组件中与权限提升相关的高风险漏洞。云原生场景下的权限提升往往涉及多类成因的叠加与链式利用。本文将权限提升漏洞界定为: 攻击者利用应用或系统缺陷, 在一定前置条件下获得超过其当前权限级别的访问能力, 从而引发越权操作或影响范围扩大的风险。

筛选规则以“权限提升/越权/逃逸相关漏洞”为主, 主要包括:

- **关键词约束**: 在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词;
- **严重性阈值**: 采用 CVSS v3.x 评分, 并以 ≥ 5.0 的高危与严重漏洞为主要对象;
- **人工复核**: 对候选条目结合公告原文、厂商通告与社区讨论进行复核, 剔除重复或与权限提升无关的记录, 并补全攻击前提、影响面与可用缓解信息。

最终保留的结构化字段包括 CVE 编号、漏洞摘要、披露时间、受影响组件与版本范围、修复版本、CVSS 评分、参考链接，以及与修复相关的补丁提交记录等。

(3) CWE 标注与分布统计。为刻画漏洞成因层面的共性，本文对样本进行 CWE（Common Weakness Enumeration）标注。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。本文依据公开映射与公告信息提取每个 CVE 的 CWE 标签，并统计不同 CWE 的出现频次。

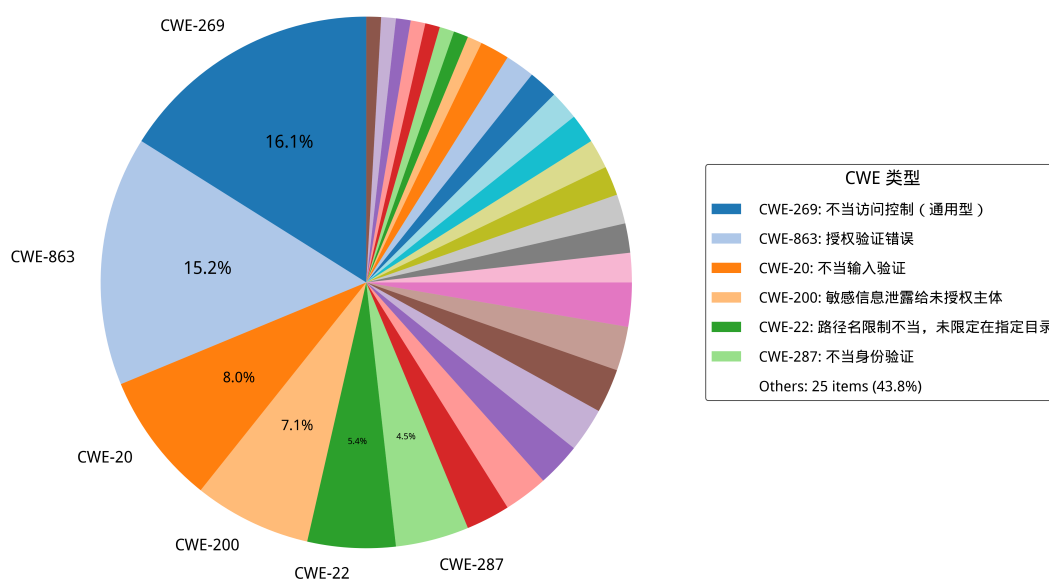


图 3-1: Kubernetes 权限提升 CVE 对应的 CWE 类型占比

需要指出的是，CWE 在组织形态上并非严格“正交、互斥”的分类体系：同一漏洞可能同时对应多个弱点条目（例如同时涉及授权缺陷与输入校验缺陷），且弱点条目在抽象层次上存在从具体机制到高层概念的混用，从而导致类别之间出现交叉。样本统计同样体现了这一现象：访问控制类弱点（如授权不当、权限管理不当等）与输入处理类弱点在部分漏洞中呈现共现。该结果表明，单独依赖 CWE 难以为防控措施选择提供唯一且稳定的指引，因而有必要引入面向防控动作的成因分类框架。

除整体分布外，进一步统计两级成因分类框架与 CWE 标签的对应关系：将每个漏洞样本同时标注为（成因一级类，成因二级类）以及一个或多个 CWE 条目，并按共现频次进行聚合。图3-2 给出了成因一级分类与 Top CWE 的共现热力图，用以刻画“不同成因类别中更常见的弱点类型”。

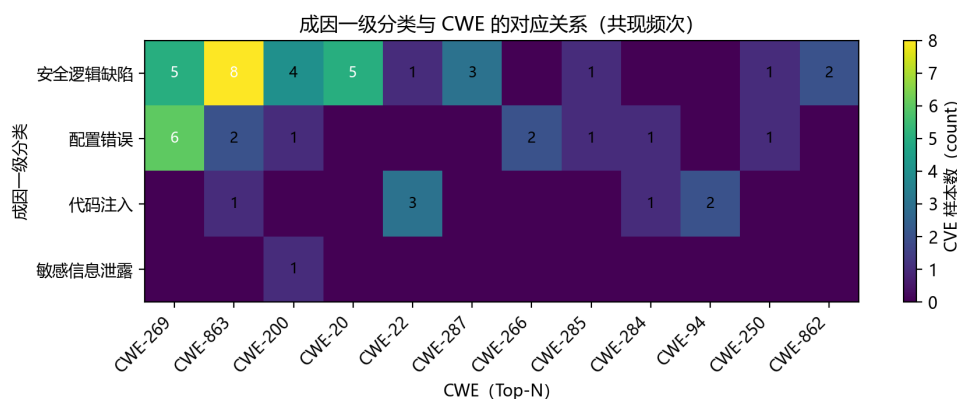


图 3-2: 成因一级分类与 CWE 的对应关系 (Top CWE 共现频次热力图)

统计结果显示: 其一, 安全逻辑缺陷与访问控制/授权相关 CWE (如 CWE-863、CWE-269、CWE-20 等) 具有更高的共现频次, 提示该类漏洞往往同时涉及“授权判断错误/缺失”与“输入/边界检查不足”等复合因素; 其二, 配置错误与权限管理/访问控制相关弱点 (如 CWE-269、CWE-266、CWE-732) 关联更为显著, 反映出 RBAC 绑定过宽、默认配置不安全等治理问题; 其三, 代码注入类与路径遍历、命令/代码注入等条目 (如 CWE-22、CWE-94) 对应更集中; 其四, 同一 CWE 跨多个成因类别出现 (例如访问控制相关条目在安全逻辑缺陷与配置错误中均可见), 与前述 CWE 的非正交性相一致, 亦从侧面说明两级成因分类在防控语义表达上具有更稳定的指示作用。

为增强统计结果的可解释性, 选取与权限提升相关且出现频次较高的典型 CWE 条目, 并给出样本中的代表性 CVE 作为对照, 以提升论证的可复核性。

CWE-863 (授权不当) 该类弱点指授权判断存在错误或缺失, 从而在特定条件下可能导致未授权主体访问受限资源。其风险通常表现为访问控制决策与预期策略不一致, 进而引发越权读取或操作。样本中, **CVE-2019-11247 (Kubernetes)** 涉及 kube-apiserver 在处理集群范围自定义资源请求时的授权判定偏差: 在特定请求构造下, 授权检查可能未按预期生效, 从而引入越权访问风险 (成因标签: 安全逻辑缺陷/访问控制缺陷)。

CWE-269 (权限/访问控制管理不当) 该类弱点强调权限配置或权限边界治理不当, 可能使主体获得超出最小权限原则的能力, 常见于 RBAC 角色绑定过宽、默认权限不安全等治理问题。其风险通常表现为“高权限能力”被不恰当地授予或可被间接获取, 从而为权限提升链路提供基础条件。样本中,

CVE-2022-29179 (Cilium) 体现为在权限配置与隔离边界治理不足的背景下，一旦攻击链达到容器内高权限，影响范围可能进一步扩大并引入权限提升风险（成因标签：配置错误/权限配置过宽）。

CWE-22 (路径遍历) 该类弱点指对路径/目录边界的限制不足，可能使攻击者借助相对路径等手段访问受限目录之外的文件或内容。其风险通常体现为敏感配置泄露、策略文件被非法读取或覆盖，并进一步影响权限边界。样本中，**CVE-2022-24730 (Argo CD)** 体现为对文件路径的约束不充分，并与访问控制缺陷叠加：在特定条件下，攻击者可能访问本应隔离的文件或配置，从而为后续权限提升创造条件（成因标签：代码注入/路径遍历）。

CWE-287 (认证不当) 该类弱点指认证流程存在缺陷（例如校验缺失、逻辑错误或可被绕过），从而使身份校验在特定条件下可能被规避。其风险通常表现为未认证主体获取会话或身份上下文，进而触发后续授权绕过与权限提升。样本中，**CVE-2022-29165 (Argo CD)** 涉及认证校验链路缺陷，未认证用户在特定条件下可能实现身份冒充，进而引发越权访问与权限提升风险（成因标签：安全逻辑缺陷/身份验证绕过）。

(4) 面向防控的成因分类框架。为解决“根因标签不稳定/不互斥”对策略生成的影响，本文提出面向云原生权限提升漏洞的两级成因分类框架，并将每个漏洞映射为（一级类，二级类）标签：

- **一级分类**刻画成因大类（如安全逻辑缺陷、配置错误、代码注入、敏感信息泄露等），用于提供高层风险语义与控制域提示；
- **二级分类**进一步细化为更贴近防控动作选择的类型（例如授权验证不完善、默认不安全配置等），用于将漏洞分析结果对齐到可操作的控制点（RBAC 最小权限、准入控制、网络隔离、运行时策略与审计等）。

在策略生成阶段，分类结果作为提示词中的结构化约束输入之一，使大模型在“漏洞语义 → 控制域 → 可执行动作序列”的推导链条中更容易选中恰当的防控工具，并减少泛化建议与无关噪声，从而提升策略的可部署性与关键性。具体分类定义如下：

1. 配置缺陷：应用或基础设施的配置不当导致的安全漏洞，包括：

- **过度权限配置：**Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被

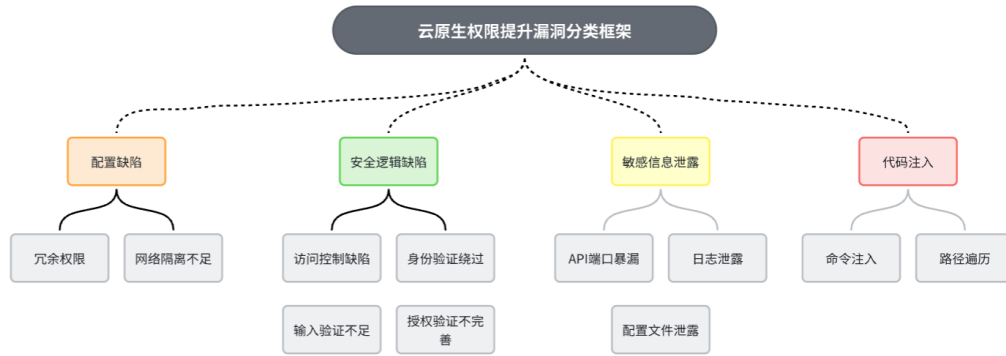


图 3-3: Kubernetes 容器化应用权限提升攻击方式分类

证明可被用于接管集群关键资源 [6, 7]。

- 网络隔离不足：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限上下文。

2. 安全逻辑缺陷：应用代码中的安全机制设计或实现不当，包括：

- 访问控制缺陷：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限验证；
- 身份验证绕过：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- 授权验证不完善：权限检查在某些操作路径或边界条件下缺失。

3. 敏感信息泄露：重要的身份、密钥或权限信息意外暴露，包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：日志文件包含敏感信息（如 token、密钥、命令历史）；
- 配置文件泄露：配置文件包含硬编码的凭证或密钥。

4. 代码注入：应用对用户输入的处理不当，允许注入恶意代码，包括：

- 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
- 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的文件。

3.3.2 知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ ，并以嵌入式模型构建一个可检索的向量索引，以支持后续“按需取证”的上下文供给。

(1) 材料收集与最小化预处理。对每个漏洞关联的开源应用仓库，收集

代码、配置与文档等可读文本；同时纳入与漏洞相关的安全公告、Issue/PR 讨论与修复提交信息。对文本进行最小化规范化（统一换行与空白），并过滤超长或二进制内容，以降低噪声与资源开销。

（2）结构化分块。为兼顾证据粒度与可复核性，本文对不同材料采用差异化分块：对代码/配置按行切分并尽量对齐函数或语义边界；对文档按段落与窗口切分并保留适当重叠，以减少跨块语义断裂。

（3）向量化与索引。对每个文本块使用嵌入式模型生成向量表示（本文实验使用 `sentence-transformers/all-mpnet-base-v2`），并以 FAISS 构建近邻索引以支持相似度检索。为保证可追溯与可复现，索引与元数据同步持久化：包括向量索引文件、块级元数据（来源路径/链接、块 ID、块类型与范围等）以及构建配置（分块参数与过滤阈值）。

上述流程得到的向量知识库为后续策略生成提供了“可追溯证据”的供给能力：模型在推导控制措施时可以检索到与漏洞相关的关键事实与上下文片段，并通过来源标识完成快速定位与复核。

3.4 策略生成

3.4.1 方法概览

策略生成阶段旨在根据漏洞实例 v 与安全上下文 c 形成候选策略集合 Π_v 。如图3-4 所示，本文将该过程划分为三个相互衔接的子过程：**安全上下文构建（RAG）、基于漏洞分析思维链的策略生成（CoT）与基于评审反馈的策略优化（Feedback）**。其中，前者负责形成可追溯的证据输入；中间环节在结构化约束下完成“分析—选型—生成”的闭环；后者通过评审与二次优化降低噪声与误报风险，提升策略的可部署性与一致性。图3-4 给出了方法的流程示意。

3.4.2 安全上下文构建

安全上下文构建以“证据可追溯”为目标，将漏洞相关信息组织为 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ 的证据集合。一方面，系统从应用知识库与安全材料中检索并汇聚候选证据（例如源代码、配置、文档、Issue、PR 与安全公告等），以兼顾覆盖度与召回精度；另一方面，引入**安全知识智能体**对检索结果进行二次整理：识别与 CVE 相关的关键事实（触发条件、受影响组件、权限边界与可

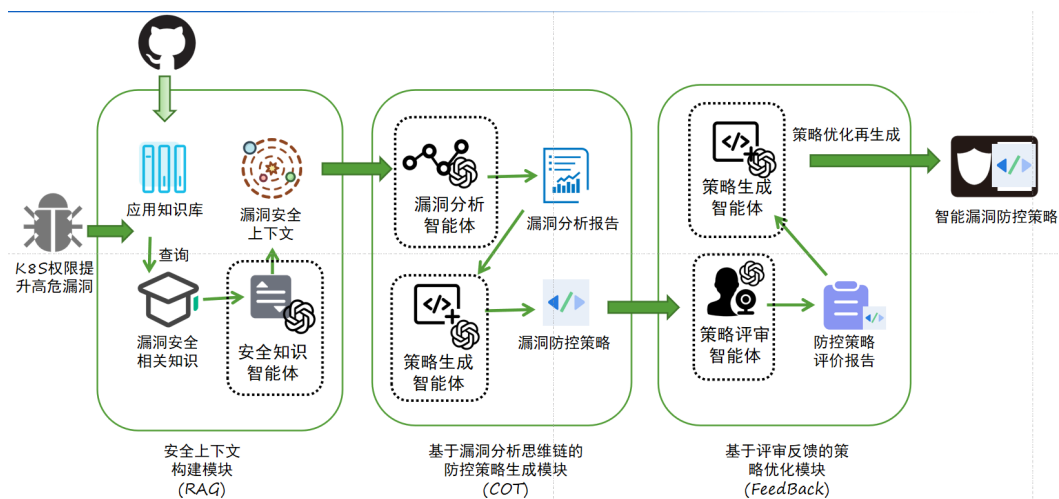


图 3-4: 智能安全策略生成框架

控点等)，并过滤与漏洞无关或重复的噪声片段。为支持后续复核与差异化处理，每条证据均保留来源标识与必要元数据；同时对冗余、超长或低质量文本进行裁剪与去重，避免上下文过载影响后续推导。

3.4.3 基于漏洞分析思维链的策略生成

该步骤在结构化约束下完成“漏洞分析—技术选型—策略生成”的智能 workflow，并显式化关键中间结论，便于检查与复核。与仅输出自然语言建议不同，本文将中间产物与最终策略均约束为固定的 JSON 结构，从而降低信息缺失与表述歧义对交付的影响。

首先，**漏洞分析智能体**基于漏洞摘要与安全上下文 c 输出结构化分析结果，覆盖漏洞类型、攻击向量、影响范围与风险等级，并给出与本文分类体系一致的风险类别。随后，**策略生成智能体**在提示词模板与思维链 (CoT) 引导下，以分析结论为约束完成控制手段选择与动作序列生成：结合平台可用能力在身份与权限、网络隔离、运行时安全与策略执行等控制域中确定优先级与实施路径，并将其映射为最终策略 JSON；当信息不足以确定时，以 (info needed) 显式标注缺失信息并给出必要假设。

为保证与本文形式化定义一致，策略内容采用 layer/method/actions 的表达方式：其中 layer 与 method 描述防控所在安全域与方法类别；actions 以可执行步骤序列承载具体控制措施。在工程化输出中，actions 可等价展开为策略 JSON 的 mitigation.action 字段，采用“1.. 2..”的步骤格式，并包含验证要点与回滚边界；当需要给出 YAML/CLI 片段时，统一使用转义换行 \n 以保持 JSON 合法性。

- **layer 字段**：防控所在的技术层面，用于分类防控策略所涉及的安全域，包括但不限于：
 - 身份与权限：RBAC、认证强化、ServiceAccount 管理等；
 - 网络隔离：防火墙规则、NetworkPolicy、流量隔离等；
 - 运行时检测：容器监控、系统调用检测、入侵检测等；
 - 审计与日志：日志记录、事件审计、合规监控等。
- **method 字段**：防控方法的摘要或分类，用于快速理解防控策略的整体思路，例如：
 - 网络隔离与认证强化；
 - 权限最小化与配置加固；
 - 检测与阻断相结合。
- **actions 字段**：具体可执行的操作序列，遵循定义 3.1，包含：
 - 对应的安全工具或修改对象（如 Cisco DNA Center、Kubernetes RBAC 配置）；
 - 具体的执行步骤，包括命令、参数、验证方式；
 - 副作用评估与回滚方案。

为提高交付可用性，本文对候选策略施加基本质量要求：操作序列应可在短时间内启动；至少覆盖一个明确的防控层次；包含可验证的执行步骤；对潜在副作用与回滚边界作出清晰说明。

候选策略集合 Π_o 的生成可采用多策略并行方法：对同一漏洞产生多个结构化策略对象，以探索不同的防控层次和防控方法。由于所有候选共享统一的 schema 结构（layer、method、actions），它们可直接进入后续的评价与排名流程。

在交付前，生成结果需进行规范化处理：统一各字段的格式、补齐缺失的操作步骤、清理冗余信息，并在数据缺失或不完整处显式标注假设条件和依赖的前置条件。

3.4.4 基于评审反馈的策略优化

该步骤通过“评审—反馈—再生成”的闭环对候选策略进行质量控制，以降低噪声与误报风险，并提升可部署性与一致性。具体而言，系统可并行生成多份候选策略形成 Π_o ，随后由策略评审智能体对每份候选进行结构化评审，提炼有效控制点，指出潜在噪声来源（如误报概率高、运维负担过重或依赖

条件不成立的配置)，并给出上线前的验证需求与缺失信息清单。评审结论作为约束输入交由**策略优化智能体**进行二次整合：合并多份候选中被验证更可行的措施，剔除不可行或噪声过高的建议，补齐关键前置条件与回滚方案，并输出最终的优化策略。

在实现层面，本文将该过程与格式校验结合：若生成结果不满足 JSON 合法性或 schema 约束，则触发重试提示并要求按既定结构重新生成；优化阶段同时记录“改进主题—改进内容—来源候选”的对应关系，以便在复核时追踪优化依据与变化点。

3.5 质量评价与筛选

该阶段对候选策略集合进行结构化评价，输出门槛评分 $s(\pi)$ 、多维质量向量 $q(\pi)$ 和策略排名 $R(\Pi_v, \text{dim})$ ，遵循“结构化输入 → 规则化检查 → 证据支撑的结论”的流程，以提高结论的可验证性。

3.5.1 门槛评分与质量评估

评分函数 $s: \Pi \rightarrow [0, 1]$ 用于判定策略是否满足最低可用性要求。该评分关注可部署性和风险降低效果。典型评价流程包括：

评价指标。建立可核查的检查清单，包括：

- 完整性检查（关键字段是否齐全，信息缺失是否标注）
- 可部署性检查（是否给出具体步骤、验证方法和回滚边界）
- 一致性检查（策略与安全上下文的一致性，是否包含无依据的断言）
- 约束合规检查（是否符合临时防控的适用范围）

评分聚合。检查项结果通过加权和进行聚合：

$$s(\pi) = \text{clip} \left(\sum_{j=1}^m w_j \cdot r_j(\pi), 0, 1 \right)$$

其中 $r_j(\pi)$ 为第 j 项的评分， w_j 为权重。部分关键项可设置为硬约束，即任何关键项失败则整体评分低于阈值。

流程控制。设定阈值 τ ，当 $s(\pi) < \tau$ 时策略转入反馈优化流程重新修订，当 $s(\pi) \geq \tau$ 时进入后续排名和筛选。

质量向量。对通过门槛的策略，进一步用三维向量刻画其特性：

$$q(\pi) = (E(\pi), I(\pi), D(\pi))$$

其中：

- $E(\pi)$: 有效性, 即覆盖关键攻击路径和降低风险的程度
- $I(\pi)$: 干扰程度, 即对业务运营、误报风险和运维复杂度的影响
- $D(\pi)$: 可部署性, 即步骤清晰度、可操作性和验证回滚能力

每个维度通过可观测指标聚合为 $[0, 1]$ 分数, 同时记录评分理由与证据来源。

3.5.2 多维排名

排名算子 $R(\Pi_v, \text{dim})$ 在指定维度上对候选策略进行排序:

$$R(\Pi_v, \text{dim}) \rightarrow (\pi_{(1)}, \pi_{(2)}, \dots, \pi_{(k)})$$

排名过程包括:

1. **过滤**。移除评分低于阈值 τ 的候选, 使后续排名的策略均可部署。
2. **排序**。按指定维度 (有效性、干扰程度或可部署性) 的评分降序排列。
3. **并列处理**。当分数接近时, 引入次级准则 (如降低干扰程度作为次级准则)。
4. **结果说明**。输出排名序列及每个策略的三维评分、关键优缺点和支撑证据。

4 漏洞防控系统

本章给出第3章技术方案的系统化落地形态，重点说明模块划分、数据流与运行流程，使后续实现与复现实验具备清晰的工程参照。

4.1 系统架构

系统采用“数据准备—推理生成—评价筛选—发布回滚”的分层架构，可抽象为三类平面：

- **数据平面**：负责多源数据采集、分块、去重、向量化与索引维护，提供可追溯的上下文检索能力。
- **控制平面**：负责任务编排与流程控制（RAG→CoT→Feedback→Evaluation），统一 schema 校验、重试策略、日志与结果落盘。
- **发布平面**：对通过门槛的策略进行发布、验证、灰度与回滚，并与集群变更流程（CI/CD 或运维流程）对接。

4.2 核心模块

4.2.1 证据与知识库服务

该模块面向“证据可追溯”目标，完成应用仓库与安全材料的收集、分块、索引与元数据维护，输出带来源标注的证据集合 $c = \{(t_i, \text{src}_i)\}$ ，并提供检索接口支持 RAG。

4.2.2 策略生成与优化服务

该模块以漏洞实例 v 与上下文 c 为输入，执行分阶段推导并输出候选策略集合 Π_v ：

- **RAG**：检索并汇聚与漏洞相关的证据片段，进行裁剪与去噪，保留来源标识。
- **CoT**：先输出结构化漏洞分析（类型、攻击向量、影响范围、风险类别），再在 layer/method/actions 约束下生成动作序列。
- **Feedback**：对候选进行评审，依据“缺失信息—前置条件—副作用—回滚边界”反馈进行二次优化。

同时集成 JSON/schema 合法性校验，确保产物可用于后续自动化处理。

4.2.3 质量评价与审阅

该模块实现门槛评分 $s(\pi)$ 与三维质量向量 $q(\pi) = (E, I, D)$ 的规则化检查，并在三个维度上生成排名 $R(\Pi_v, \text{dim})$ 。评价结论同时输出理由、证据来源与缺失信息清单，支持 human-in-the-loop 复核与决策。

4.2.4 策略发布与回滚

该模块将最终策略映射到可执行的控制手段（如准入策略、NetworkPolicy、运行时检测规则等），并提供“验证—灰度—回滚”的发布流程：上线前对配置语法、目标资源范围与冲突策略进行检查；上线后结合告警与观测数据评估副作用，必要时按预设边界回滚。

4.3 工作流程

端到端流程如下：

1. 输入漏洞实例 v （CVE 描述与元数据），触发上下文检索构建 c 。
2. 基于 v 与 c 并行生成多份候选策略形成 Π_v ，并进行 schema 校验与规范化。
3. 对 Π_v 执行门槛评分与三维评价，过滤未达标候选并对达标候选进行维度排名。
4. 评价报告作为反馈约束触发候选优化；达标策略进入人工复核与发布决策。
5. 发布到集群并执行验证与观测；若出现副作用或误报，按回滚边界撤销或调整策略。

4.4 部署与运行

系统运行依赖包括：向量索引存储（FAISS 及元数据文件）、大模型调用接口与密钥管理、以及与集群交互的凭据与权限控制。为保证可运维性，建议记录每次生成的输入、检索证据摘要、候选策略、评分与排名结果，并对关键失败点（检索为空、schema 不合法、部署失败、回滚触发）进行可观测化告警。

5 实验研究

5.1 实验设置

实验以 55 个权限提升漏洞实例为对象，在统一的提示与输出 schema 下对五个大模型（Qwen3、DeepSeek、GPT-5、Grok、GLM-4.6）生成候选策略集合 Π_v 。为验证关键模块的贡献，设置三项消融：移除检索增强（w/o_rag）、移除思维链分阶段推导（w/o_cot）、移除基于评审反馈的迭代优化（w/o_feedback），其余流程保持一致。

为对比传统静态安全工具，选取 kubelinter、kubescape 与 kubesec 作为基线，输出风险/合规报告后人工评估其对策略生成的启发性（能否指向与漏洞路径相关的高风险配置、是否给出可落地的修正建议）。

评审阶段由上述五个模型同时作为“评审者”对 Π_v 进行排名式评分。评价指标采用三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，分别对应有效性、无干扰性与可部署性；优先使用相对排序降低不同模型打分尺度差异带来的偏差。

5.2 评价流程与指标

候选策略先通过门槛评分 $s(\pi)$ 的完整性与可部署性检查，再进入三维度评分与排名：

- 完整性：关键字段与证据引用是否齐全，缺失信息是否显式标注。
- 可部署性：是否给出具体步骤、验证方式与回滚边界，JSON/YAML 语法是否符合 schema。
- 一致性：结论是否与安全上下文匹配，是否存在无依据断言。
- 干扰性与副作用：对业务运维的影响、误报风险与额外复杂度。

通过阈值后，对 $E/I/D$ 三个维度分别排序生成 $R(\Pi_v, \text{dim})$ ，并附带关键优缺点与支撑证据以便复核。

5.3 结果与观察

本节优先从不同方法配置（含消融）出发，聚焦“有效性”维度说明全量方法（all）的提升与稳定性；随后给出生成模型分层差异；再讨论评审模型偏好差异；最后补充三维度（ $E/I/D$ ）的整体对照。

5.3.1 有效性：消融验证与 all 方法优势

图5-1 给出跨评审者汇总后的方法配置平均排名：全量方法（all）在有效性上整体领先，且与消融版本拉开稳定差距。

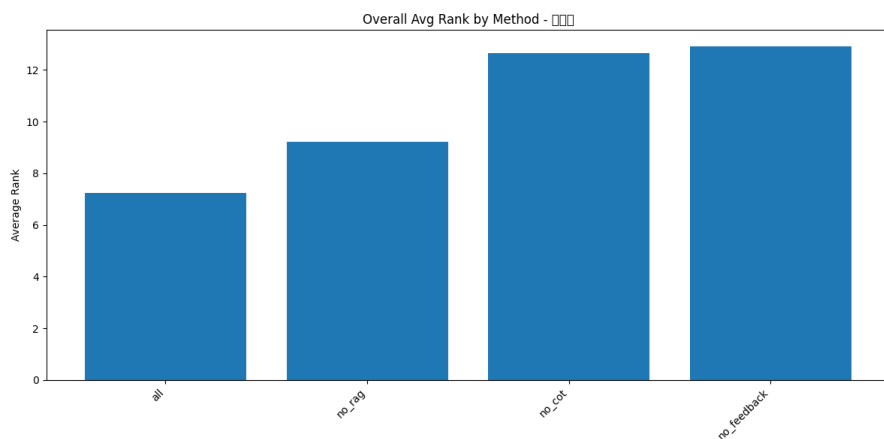


图 5-1: 有效性维度下，不同方法配置的平均排名式评分结果（跨评审者汇总）。

为进一步检验 all 方法优势是否对评审者与生成模型稳健，分别从“评审视角聚合”和“生成视角聚合”给出消融对比，如图5-2 与图5-3 所示。

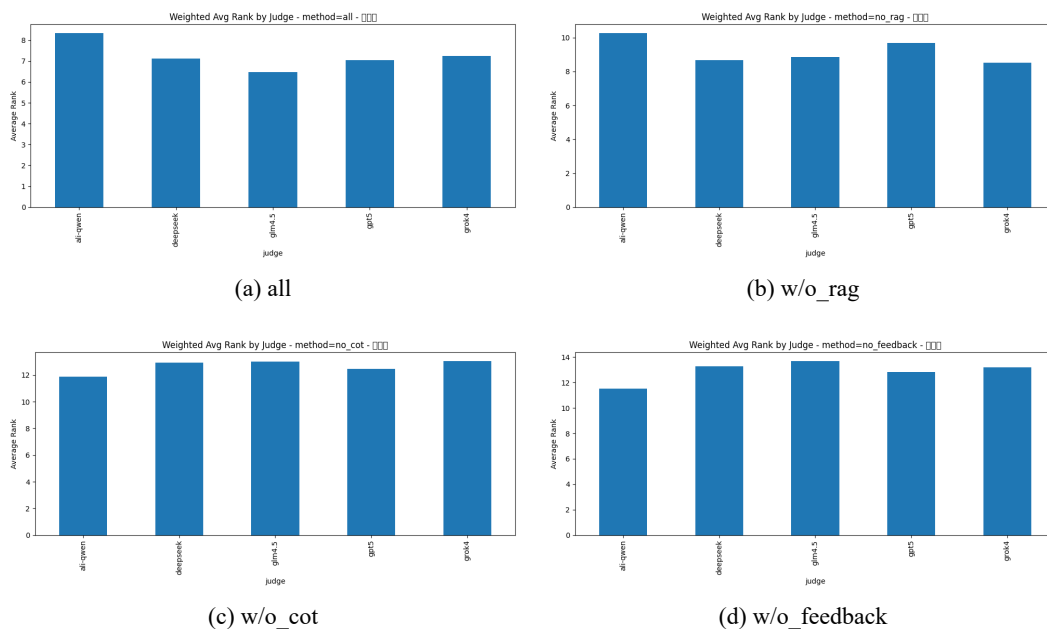


图 5-2: 有效性维度下，按评审模型聚合的消融对比（2×2 网格）。

在更细粒度的结构层面，热力图可直观看到不同配置下的“结构性差异”与离群现象。图5-4 给出四种方法配置在有效性维度的热力图对照。

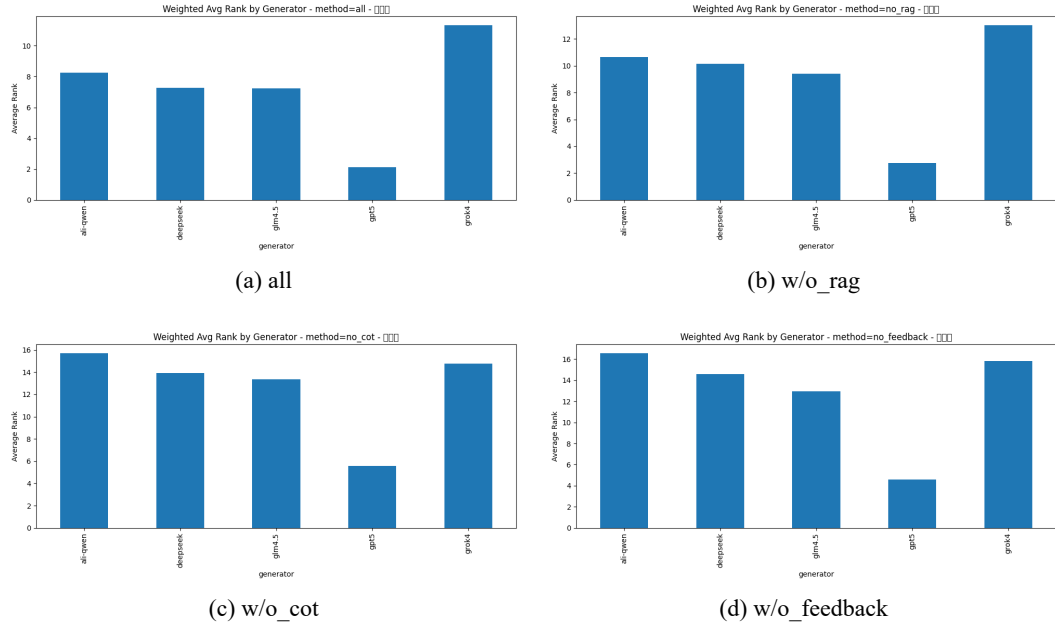


图 5-3: 有效性维度下, 按生成模型聚合的消融对比 (2×2 网格)。

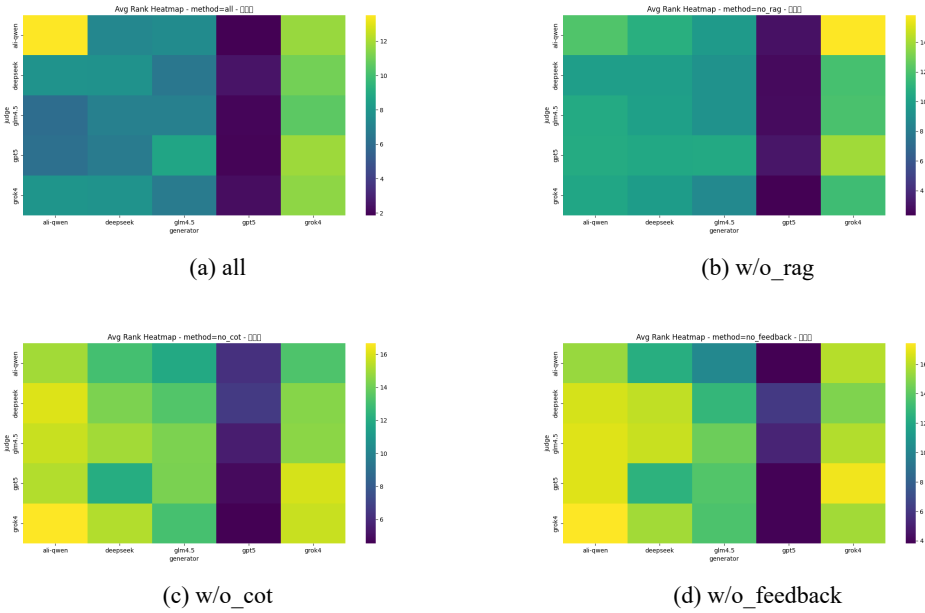


图 5-4: 有效性维度下, 四种方法配置的热力图对照 (2×2 网格)。

5.3.2 有效性：生成模型分层 (profile)

在确认 all 方法有效性优势后, 进一步关注“生成模型能力差异”是否导致分层现象。图5-5 给出不同生成模型在有效性维度的平均排名式评分结果 (跨评审者汇总)。

为比较不同生成模型在“方法配置贡献结构”上的差异, 图5-6 将五个生成模型的结果以 3+2 网格并列展示 (图题中统一采用论文表述的模型名称)。

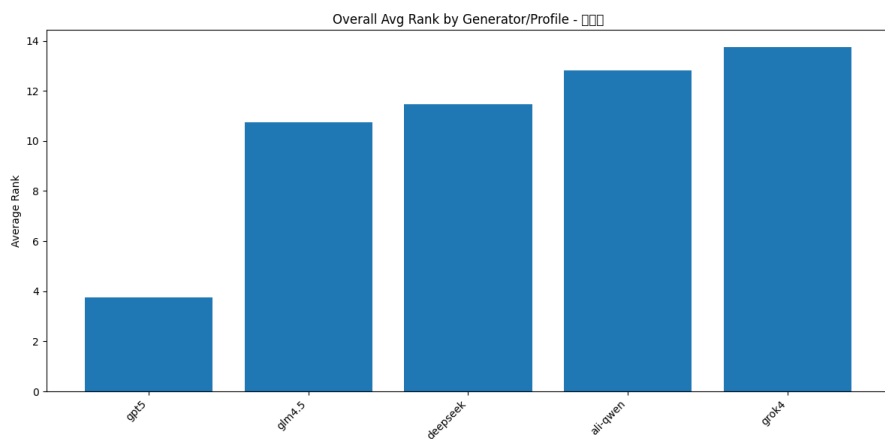


图 5-5: 有效性维度下，不同生成模型的平均排名式评分结果（跨评审者汇总）。

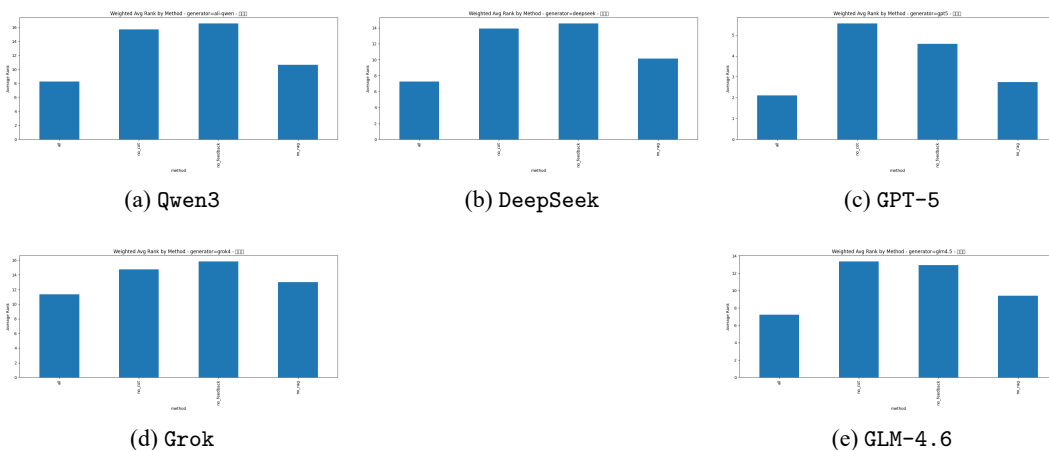


图 5-6: 有效性维度下，不同生成模型的“方法配置贡献结构”对照（3+2 网格）。

图5-7 进一步给出更紧凑的分组条形图对照，便于快速横向扫读不同生成模型下的相对差距。

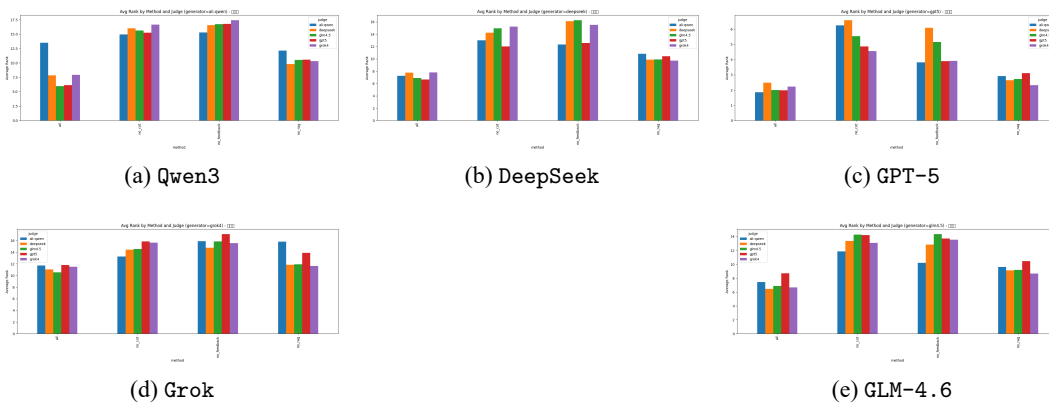


图 5-7: 有效性维度下，不同生成模型的分组条形图对照（3+2 网格）。

5.3.3 有效性：评审模型偏好差异

评审模型之间存在偏好差异。图5-8 先给出有效性维度下“按评审者拆分”的汇总结果。

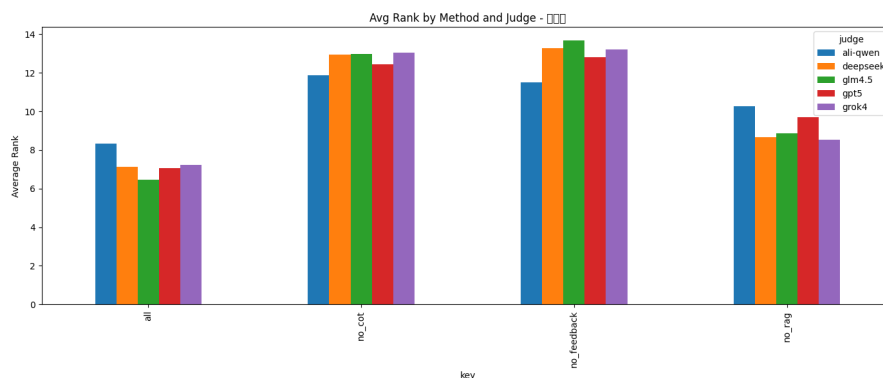


图 5-8: 有效性维度下，不同评审模型的排名式评分汇总。

为更直观地观察评审偏好结构，图5-9 给出五个评审模型对应的热力图并列对照。

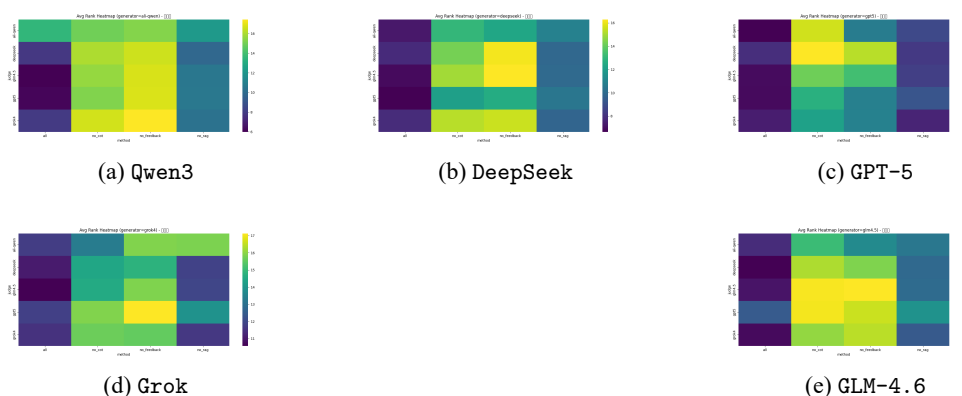


图 5-9: 有效性维度下，不同评审模型的偏好结构热力图对照（3+2 网格）。

5.3.4 三维度总览：有效性、无干扰性与可部署性

在“有效性优先”的分析后，本小节补充三维度（ $E/I/D$ ）的整体对照，以便观察方法配置与生成模型的潜在权衡关系。

小结

- 有效性维度上，all 方法在跨评审者汇总与多种聚合视角下均保持领先（图5-1–图5-4），表明 RAG、CoT 与 Feedback 的组合对“策略能否真正命中漏洞路径”具有稳定增益。

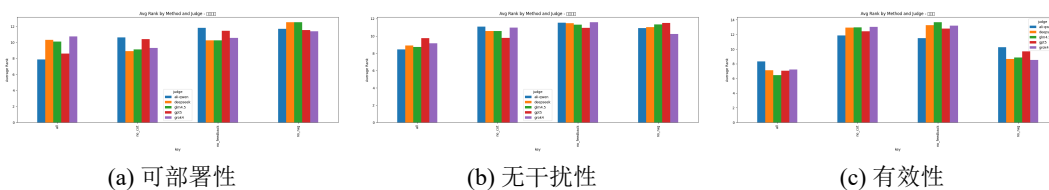


图 5-10: 不同评审模型下的单维度排名式评分结果（三维度并列）。

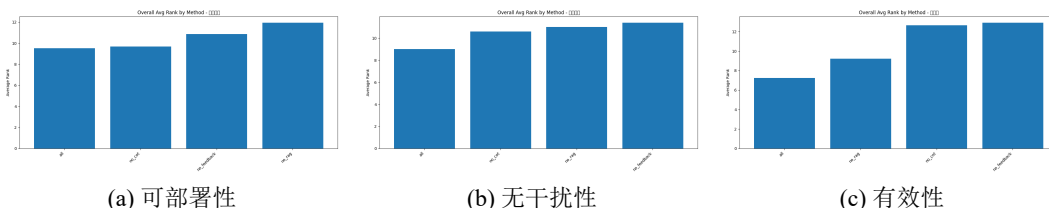


图 5-11: 不同方法配置在三个维度上的平均排名式评分结果（跨评审者汇总，三维度并列）。

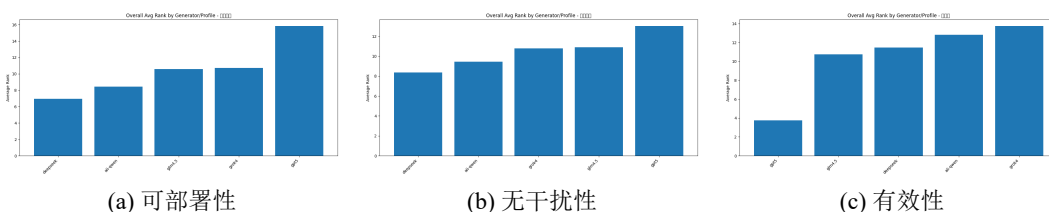


图 5-12: 不同生成模型在三个维度上的平均排名式评分结果（跨评审者汇总，三维度并列）。

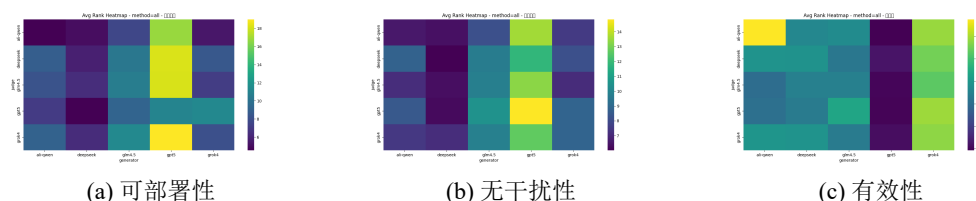


图 5-13: 全量方法配置（all）下三个维度的热力图总览（三维度并列）。

- 不同生成模型存在能力分层与偏好差异，但 all 方法的提升在各 profile 下整体可复现（图5-5–图5-7）。
- 评审模型之间存在偏好差异（图5-8–图5-9），使用相对排序与跨评审聚合可降低尺度不一致的影响，适合作为反馈闭环触发的约束信号。
- 三维度总览显示：方法与生成模型在 $E/I/D$ 上存在权衡关系，需在可部署性与干扰性约束下进一步筛选与复核（图5-10–图5-13）。

5.4 讨论

结果表明：在证据约束与结构化 `schema` 下，大模型可稳定产出可复核的候选策略；同时，分阶段推导与反馈优化对降低噪声、补全前置条件与回滚方案尤为关键。与传统静态工具相比，生成式流程能更快给出跨控制域的操作序列，但仍需结合工具扫描结果与人工复核以确保落地可行性。未来可结合运行时验证与自动化回归测试，将评价闭环前移到部署前的模拟环境。

6 案例分析

本章面向“可复核的快速缓解”目标，给出一个典型漏洞从输入到交付的端到端落地示例写法，用于将第3章方法与第4章系统串联起来。由于不同项目与集群环境差异较大，本章聚焦于流程与产物形态：证据如何组织、策略如何表达、如何评价与如何验证。

6.1 案例概述

选择实验集中的一个权限提升漏洞实例 v 作为案例输入，包含 CVE 标识、漏洞描述 d 与元数据 m （严重性、受影响组件与版本等）。案例目标是在补丁未及时可用的窗口期内，生成可部署的临时防控策略，并明确其适用边界、验证方式与回滚方案。

6.2 证据构建与上下文

基于知识库服务对目标开源应用仓库与漏洞相关材料进行证据汇聚，形成带来源标注的上下文 $c = \{(t_i, \text{src}_i)\}$ ：

- 源代码与配置：与权限边界、鉴权逻辑、运行参数相关的代码/配置片段。
- 文档与安全公告：部署方式、默认权限、暴露端口与修复建议。
- Issue/PR/Commit：漏洞讨论、补丁差异与已知副作用。

为避免上下文噪声影响推导，证据需在保留来源的前提下去重、裁剪，并将缺失信息显式标注为 (info needed) 供后续复核。

6.3 策略生成与优化过程

在统一 schema 约束下并行生成多份候选策略形成 Π_v ，每份策略以 layer/method/action 表达：

- **候选生成**：先输出结构化漏洞分析（攻击向量、影响范围、可控点），再映射到具体控制域（准入、网络隔离、运行时检测与审计等）形成动作序列。
- **评审反馈**：对候选逐条检查“步骤可执行性、验证与回滚边界、与证据一致性、潜在干扰性”，输出缺失信息清单与修改建议。

- **二次优化：**根据反馈合并可行措施、剔除噪声较高建议，补齐前置条件与回滚方案，并通过 JSON/schema 校验保证格式可用。

6.4 部署与验证

最终策略在发布前后需完成验证闭环：上线前检查策略目标范围与冲突；上线后通过对业务指标与安全告警的观测评估有效性与副作用，并在触发异常时按回滚边界撤销策略或降低强度（例如从阻断改为告警）。上述过程与产物（证据引用、评分理由、变更记录）应一并留存，以便复盘与审计。

7 结论与展望

7.1 结论

本文面向云原生权限提升漏洞的快速缓解需求，提出了以“证据可追溯、结构化产物、质量闭环”为核心的策略生成框架，主要结论如下：

- 提出 `layer/method/actions` 结构化策略表示与 JSON schema 约束，结合证据来源标注，使生成结果可检查、可回溯、可复核。
- 构建开源应用知识库与多源检索增强的安全上下文，分块、去重与元数据保留提高了证据覆盖度与复现性。
- 设计“RAG+CoT+Feedback”分阶段流程，将漏洞分析、控制手段选型与策略生成显式化，并通过评审反馈迭代优化候选策略，降低噪声与误报。
- 以门槛评分与三维质量向量（有效性、无干扰性、可部署性）对候选策略进行规则化评价与排序，实验表明各模块均对交付质量有显著贡献，生成式方法较静态工具更能产出可落地的缓解序列。

7.2 展望

未来工作可从以下方向展开：

- **数据与覆盖面：**扩展漏洞与应用样本，纳入供应链依赖、运行时观测数据，增强对链式攻击场景的适配能力。
- **验证与部署自动化：**将生成策略在隔离环境中自动验证（语法检查、仿真与回归测试），缩短从生成到上线的闭环时间。
- **评审增强与人机协同：**引入更细粒度的风险标注与可解释性输出，优化评审提示与可视化，降低安全工程师的复核成本。
- **与现有工具集成：**将生成式流程与静态/动态检测工具、CI/CD 流水线结合，实现持续化的策略生成、验证与发布。

参考文献

- [1] Kubernetes. Kubernetes documentation. Available online: <https://kubernetes.io/docs/home/>, 2024. Accessed: November 18, 2024.
- [2] Cloud Native Computing Foundation. Annual survey 2022. Available online: <https://www.cncf.io/reports/cncf-annual-survey-2022/>, 2022. Accessed: November 18, 2024.
- [3] Alibaba Cloud. Alibaba container service: From serverless containers to serverless kubernetes. Available online: https://www.alibabacloud.com/blog/from-serverless-containers-to-serverless-kubernetes_596533, 2020. Accessed: November 18, 2024.
- [4] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. Role-based access control (rbac): Features and motivations. In *Proceedings of the 11th Annual Computer Security Applications Conference*, pages 241–48, 1995.
- [5] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [6] Greg O’Shea. Redundant access rights. *Computers & Security*, 14(4):323–348, 1995. ISSN 0167-4048.
- [7] Nanzi Yang, Wenbo Shen, Jinku Li, Xunqi Liu, Xin Guo, and Jianfeng Ma. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, page 3048–3062, 2023. ISBN 9798400700507. doi: 10.1145/3576915.3623121.
- [8] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 32(4):1–36, May 2023. ISSN 1049-331X.

- [9] Nicholas Pecka, Lotfi Ben Othmane, and Altaz Valani. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment. In *Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering(ICSSP)*, page 45–49, 2022. ISBN 9781450396745. doi: 10.1145/3529320.3529325.
- [10] Chang-ai Sun, Xiaoyang Han, and Yufei Gong. Kubeguard: A systematic permission-oriented risk detection approach for kubernetes applications. In *2025 IEEE International Conference on Web Services (ICWS)*, pages 855–865, 2025. doi: 10.1109/ICWS67624.2025.00111.
- [11] Aqua Security. Trivy: Universal scanner for vulnerabilities, misconfigurations, and sbom. GitHub repository, 2025. URL <https://github.com/aquasecurity/trivy>. Accessed 2026-01-06.
- [12] Anchore. Grype: Vulnerability scanner for container images and filesystems. GitHub repository, 2025. URL <https://github.com/anchore/grype>. Accessed 2026-01-06.
- [13] Anchore. Syft: Cli tool and library for generating software bill of materials. GitHub repository, 2025. URL <https://github.com/anchore/syft>. Accessed 2026-01-06.
- [14] StackRox. Kubelinter: Static analysis for kubernetes yaml files. GitHub repository, 2025. URL <https://github.com/stackrox/kube-linter>. Accessed 2026-01-06.
- [15] ARMO. Kubescape: Kubernetes risk analysis, compliance, and scanning. GitHub repository, 2025. URL <https://github.com/kubescape/kubescape>. Accessed 2026-01-06.
- [16] Zegl. kube-score: Kubernetes object analysis with recommendations. GitHub repository, 2025. URL <https://github.com/zegl/kube-score>. Accessed 2026-01-06.
- [17] Control Plane. kubesecc: Security risk analysis for kubernetes resources.

- GitHub repository, 2025. URL <https://github.com/controlplaneio/kubesecc>. Accessed 2026-01-06.
- [18] Fairwinds. Polaris: Kubernetes best practices validation. GitHub repository, 2025. URL <https://github.com/FairwindsOps/polaris>. Accessed 2026-01-06.
- [19] Bridgecrew. Checkov: Static analysis for infrastructure as code. GitHub repository, 2025. URL <https://github.com/bridgecrewio/checkov>. Accessed 2026-01-06.
- [20] Yue Gu, Xin Tan, Yuan Zhang, Siyan Gao, and Min Yang. Epscan: Automated detection of excessive rbac permissions in kubernetes applications. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 11–11. IEEE Computer Society, 2024.
- [21] Carmine Cesarano and Roberto Natella. Kubefence: Security hardening of the kubernetes attack surface. *arXiv preprint arXiv:2504.11126*, 2025.

附录 A 单位

以下内容可放在附录之内：

- (1) 正文内过于冗长的公式推导；
- (2) 方便他人阅读所需的辅助性数学工具或表格；
- (3) 重复性数据和图表；
- (4) 论文使用的主要符号的意义和单位；
- (5) 程序说明和程序全文；
- (6) 企业应用证明；
- (7) 项目鉴定报告；
- (8) 获奖成果证书；
- (9) 设计图纸；
- (10) 程序源代码；
- (11) 论文发表；
- (12) 作者简介。

这部分内容可省略。如果省略，删掉此页。

书写格式说明：

标题“附录 A 附录内容名称”样式为字体：黑体，英文用 Times New Roman 字体，居中，加粗，字号：小三，2.41 倍行距，段前 17 磅，段后为 16.5 磅。

附录正文样式为字体宋体小四，英文用 Times New Roman 字体小四，两端对齐书写，段落首行左缩进 2 个字符。1.3 倍行距（段落中有数学表达式时，可根据表达需要设置该段的行距），段前 0.1 行，段后 0.1 行，1.3 倍行距。

示例

表 A-1: 表 A.1 国际单位制的基本单位

量的名称	单位名称	单位符号
长度	米	m
质量	千克（公斤）	kg
时间	秒	s
电流	安〔培〕	A
热力学温度	开〔尔文〕	K
发光强度	坎〔德拉〕	cd

表 A-2: 表 A.2 国家规定的非国际单位制单位

量的名称	单位名称	单位符号	换算关系和说明
时间	分	min	1 min=60 s
	[小] 时	h	1 h=60 min=3600 s
	天(日)	d	1 d=24 h=86400 s
平面角	[角] 秒	"	$1'' = (\pi/648000) \text{ rad}$
	[角] 分	'	$1' = 1^\circ/60 = (\pi/10800) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
旋转速度	转每分	r/min	1 r/min=(1/60) r/s
长度	海里	n mile	1 n mile=1852 m
速度	节	kn	1 kn=1 n mile/h=(1852/3600) m/s (只用于航行)

致 谢

致谢是作者对该论文的形成作出过贡献的组织或个人予以声明的文字记载，语言要客观、准确、简短。

字数一般不超过 500 字，最多不超过一页。论文作者可以在致谢页对下列方面致谢：

国家科学基金、资助研究工作的奖学金基金，合同单位、资助或支持的企业、组织或个人；

协助完成研究工作和提供便利条件的组织或个人；

在研究工作中提出建议和提供帮助的人；

给予转载和引用权的资料、图片、文献、研究思想和设想的所有者；

其它应感谢的组织或个人。

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到博士入学止）

起止年月	学习或工作单位	备注
XXX 年 XX 月至 XXXX 年 XX 月	在 XXXX 学校 XXXX 专业攻读学士学位	
XXX 年 XX 月至 XXXX 年 XX 月	在 XXXX 单位从事 XXXX 岗位的工作	
XXX 年 XX 月至 XXXX 年 XX 月	在 XXXX 学校 XXXX 专业攻读博士学位	

二、在学期间从事的科研工作

（1）高性能低功耗深度神经网络处理器芯片设计（USTB06500093），主要参与人员，2017.11-2022.11。

（2）...

三、在学期间所获的科研奖励

（1）博士研究生国家奖学金

四、在学期间发表的论文

- [1] **Liu M K**, LIU C X, ZHANG S T, et al. Research on Industry Development Path Planning of Resource-Rich Regions in China from the Perspective of “Resources, Assets, Capital” [J]. Sustainability, 2021, 13(7): 3988-3988.
- [2] **Liu M K**, LIU C X, PEI X D, et al. Sustainable Risk Assessment of Resource Industry at Provincial Level in China [J]. Sustainability, 2021, 13(8): 4191-4191.
- [3] **刘明凯**, 张寿庭, 刘昌新, 等. 基于“三资”视角的矿山企业绿色可持续发展路径研究 [J]. 中国矿业, 2020, 29(07): 35-43.

- [4] 刘明凯, 张红艳, 王新宇. 广西有色金属产业链关联测度与发展路径规划研究 [J]. 中国国土资源经济, 2020, 33(12): 65-74.
- [5] 虎海峰, 刘明凯, 樊杰. 黄河流域经济-社会-生态耦合协调效应评价及预测 [J]. 中国管理科学. 待刊出.

盲审说明（正式写作请删除此说明）：

盲审论文需要隐藏掉所有会影响盲审结果的论文作者及其导师的信息，以便论文评阅人能够公正的进行评阅。

当学校要求提供盲审论文时，请按如下方法制作。

1) 对于论文中下列学生和导师信息。请将学生姓名、学生学号、导师姓名，依次全部替换为[本论文作者]、[论文作者学号]、[本论文导师]。论文中上述信息均需要替换，包括作者研究成果等部分的有关信息。

2) 在研究成果中，论文作者发表的文章列表中应隐去所有作者的名字，只标明论文期刊名，级别，发表年份。

3) 盲审论文，请不要填写致谢，致谢页除标题、页眉、页码外请保持空白。

4) 其他会影响盲审结果的信息，请采用类似方式处理。

5) 提交的盲审论文应为正式论文，除了上述替换后的信息外，应为可以评阅的正式论文。

6) 论文封面，请填写专业名称、论文题目等，将学号和姓名项目保持空白不填。制作盲审论文时，论文书脊、封二、题名页请删除。

替换前后将文档分别保存，以便盲审论文与其他论文分开管理。

盲审样例（正式写作请删除此样例）：

五、在学期间发表的论文：

- [1] **第一作者**. 中国矿业. 中文核心期刊.2020.
- [2] **第二作者（导师一作）**. 中国矿业. 中文核心期刊.2020.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
学位授予单位名称 *		学位授予单位代码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
作者姓名 *			学号 *	
培养单位名称 *		培养单位代码 *	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学院路 30 号	100083
学科专业 *		研究方向 *	学制 *	学位授予年 *
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席 *	答辩委员会成员		
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 () 推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				